GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

从编码智能体到自主贡献者：设计智能体软件工程的运营模式

From Coding Agents to Autonomous Contributors: Designing the Operating Model for Agentic Software Engineering



Susanna Wong

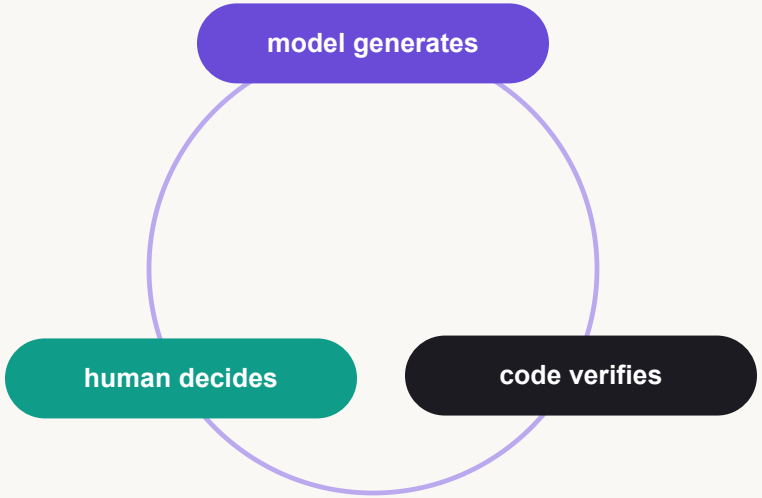
Typeform资深AI工程师，Google开发者专家（GDE）

Staff AI Engineer, Typeform，Developer Expert, Google

GOSIM SHENZHEN 2026 · AGENTIC OS & APP

From coding agents to autonomous contributors

Designing the operating model for agentic software engineering



Susanna Wong

Staff AI Engineer, Typeform

Agents write code. Contributors ship it.

The step from assistant to contributor is not a better model. It is everything around the model.

CODING ASSISTANT	AUTONOMOUS CONTRIBUTOR
STARTED BY A person at a keyboard	**STARTED BY** **An event: an issue label, a review comment**
SUPERVISED Line by line, in real time	**SUPERVISED** **Only at decision points**
WORKS IN Your checkout, your laptop	**WORKS IN** **Someone else’s repo, in CI, unattended**
PROOF OF WORK “Looks right to me”	**PROOF OF WORK** **Evidence a machine re-derives**
WHEN IT’S WRONG You notice	**WHEN IT’S WRONG** **It must notice, stop, and hand back**

The gap is the operating model: who triggers it, what it may touch, who checks it, and when it stops.

THE ONE THING TO TAKE HOME

Autonomy is a property of the system, not the model.

RULE 1

Separate generation

The model proposes.
Deterministic code decides
what is true, and re-derives
every claim it makes.

`agent output = a claim`

RULE 2

Bound the blast radius

Least privilege, short-lived
credentials, path allowlists,
enforced in code. It can never
edit its own rules.

`boundaries in code, not prompts`

RULE 3

Stop and hand back

Turn caps, round caps,
escalation counted as success.
Every exit lands at a human,
and a human merges.

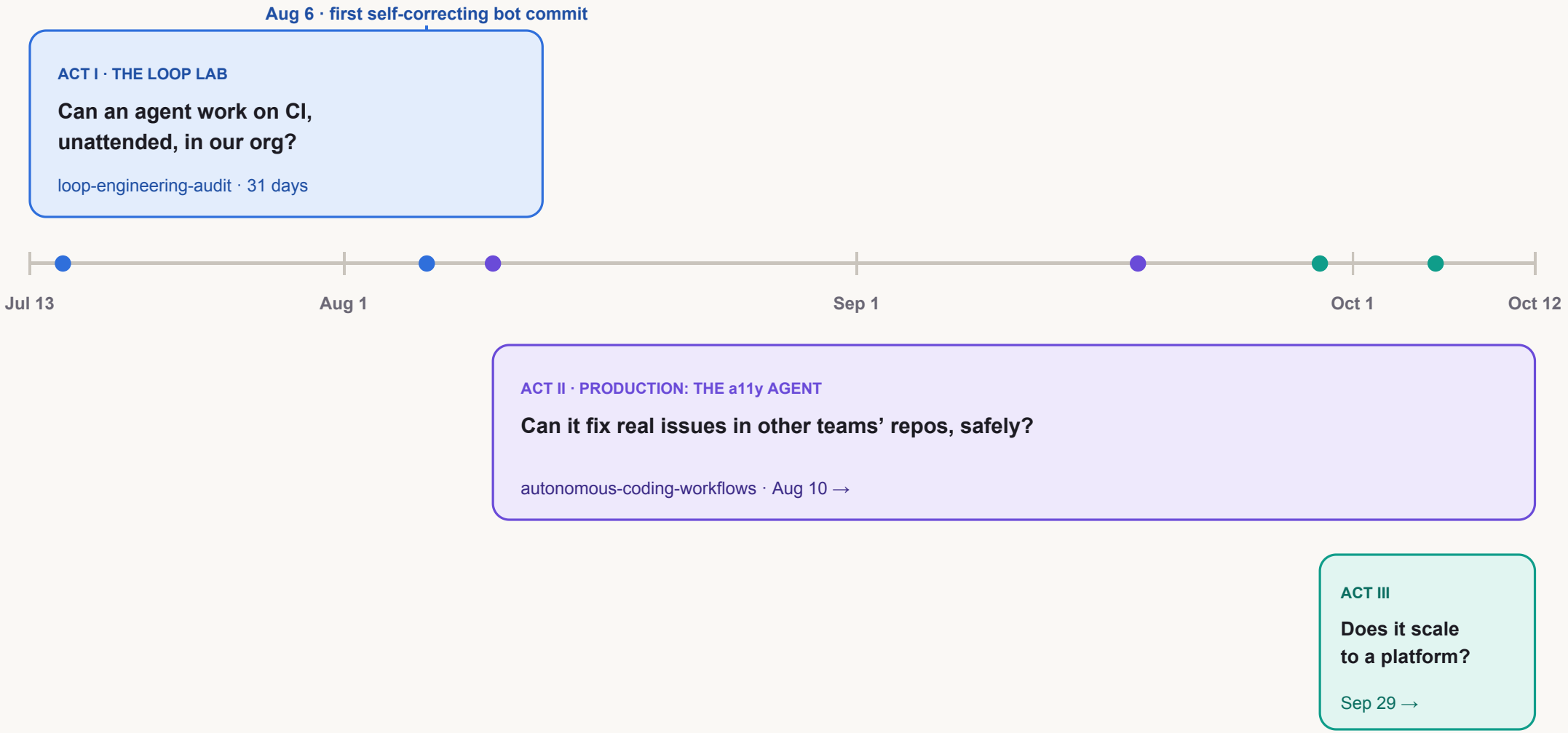
`every exit → a human`

Three agents, three months: every design decision in this talk is one of these three rules, learned the hard way.
Each slide is tagged with the rule it shows.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

Three stages, three months

Each act answered a question the previous one raised



ACT I

The loop lab

Can an agent do real work on CI, unattended, inside our org?

31
days, Jul 13 → Aug 13

78
commits

435
repos swept

24
days to a self-fixing PR

Step 1: how agent-ready are we, really?

A deterministic audit scores every repo the same way, then a human reads it



WHAT THE AUDIT SCORES

Option 1 · dependency-bump loop

"autonomy signal": dep bot, bump volume,
CI reliability (CI is the oracle) · /15

Option 2 · feature-ticket pipeline

"assistance signal": specs, backlog, tests · /15

`priority = impact × ease`

```
impact: prod criticality, throughput, toil…
complexity: blast radius, CI trust, tests…
tiers at 0.55: Strong · Invest · Quick win
```

The golden rule

Scripts collect identical facts per repo.
Every number is computed once, in Python.
Fuzzy calls (CI trust, spec quality)
stay with humans.

Source: loop-engineering-audit results/comparison.md, 8 repos, Aug 10 2026

ACT I · THE LOOP LAB

Step 2: sweep the whole org, cheaply

cross-org-audit-on-issue.yml · a funnel that never clones until the end

435 — **Tier 1a · metadata sweep**
one batched gh repo list · ~11 API calls · no clone

150 — **Tier 1b · light enrichment**
~5 core-API calls each · never /search

30 — **Rank + shortlist**
all Strong · prod + CI + Dependabot

→ 1 — **Tier 2 · deep dive**
clone-based, per repo

A human decides what graduates to Tier 2.

86% of 435 active repos scored coarse-ready · zero LLM tokens spent on the 435 → 30 funnel

THE AGENT'S ONE JOB

```
<!-- EXECUTIVE_SUMMARY -->
```

2–3 paragraphs, grounded only in numbers already in the file.
Sonnet · --max-turns 15
Read, Grep, Glob, LS, Edit

The report is untrusted data

every row derives from repo names and topics: _safe() escapes them, breaking any injected <!-- --> fence

Fail loud, never plausible

swept == 0 or enriched == 0 → exit 1. No shipping a plausible-looking empty report.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

From a skill on my laptop to an agent on CI

audit-on-issue.yml · an issue is the request, a label is the trigger, a CI runner is the agent

1 File an issue

audit-request form,
one owner/repo per line
no laptop, no setup

→

2 Add the label

“ready for development”
a person with write
access pulls the trigger

→

3 Agent runs on CI

GitHub Actions runner
claude-code-action
Bedrock via OIDC

→

4 Fresh report

every repo re-scored
from scratch, each run
markdown in results/

→

5 Draft PR back

PR link posted on
the issue · a human
reviews and merges

12 audit requests ran this way, Jul 15 → Aug 10 · most finished in 4–8 minutes, end to end

ON CI, THE BUDGET LIVES IN THE WORKFLOW FILE

`--model`	`claude-sonnet-4-6`
`--max-turns`	`40, later 25`
`timeout-minutes`	`30`
`concurrency`	`one run per issue`

Each one is a ceiling the agent can’t talk its way past.

ONE EARLY RUN · JUL 17

31 turns

$0.60 model cost

3.5 min agent time

Cheap once. But an org-wide tool runs per repo, per request, again and again. So every instruction has to earn its turns.

A turn cap is a cost ceiling you set in code, not a request you make in a prompt.

R1 · Generation ≠ verification

R2 · Bounded blast radius

R3 · Stop and hand back

ACT I · THE LOOP LAB

The pattern that stuck: the deterministic sandwich

audit-on-issue.yml · one narrow judgment call between two stages of plain CI

issue labelled “ready for development” → 4 guards before any paid step: label · issue type · write/admin actor · no existing PR

STAGE 1 · DETERMINISTIC CI · NO LLM

`parse issue → audit-repo.sh → audits.jsonl → analyze_audits.py → comparison.md + per-repo reports`

Every number, score, tier and ranking is produced here. No token → exit; every repo errored → exit, and no model credential is ever minted.

STAGE 2 · NARROW AGENT · THE ONE JUDGMENT CALL

`Read, Grep, Glob, LS, Edit · --max-turns 25 · no Bash, git or gh`

√ replaces <!-- ANALYST_NARRATIVE --> with 1–2 grounded paragraphs per shortlisted repo

✗ writes no number, no tier, no shortlist · comparison.md is read-only input

turn cost ≈ 2N + 2 for N shortlisted repos: work scales with the shortlist, not the corpus

STAGE 3 · DETERMINISTIC CI · NO LLM

`create-pull-request → DRAFT PR · add-paths: input/repos.txt, results/** · nothing else can be committed`

Rule of thumb that carried into every later agent: deterministic before judgment, cheap before expensive.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

The agent got smaller as the system got better

Same task, three versions in one week. Capability moved out of the prompt and into CI.

JUL 15 · v1

Agent does it all

40

max turns

validate input, edit the list,
run the bash pipeline, commit,
open the PR, notify
org-wide token in its Bash

Worked. Scary.

JUL 22 · a detour

Give it more rope

300

max turns

Bash(git *), Bash(gh pr *)
to “let it finish the job”
reverted the same day after a
security review

Rolled back in hours.

JUL 22 → · final

One narrow job

15–25

max turns

Read, Grep, Glob, LS, Edit
fill one marker with prose
every number from Python
no secrets in its environment

Cheap, boring, trustworthy.

Shrinking the agent’s job is what made it safe to run unattended.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

Step 3: an agent that reviews PRs

pr-review-on-ready.yml · contents: read · it can comment, never approve, merge or push

TRUSTED · GITHUB API FACTS

- changed-files.md
- ci-status.md
- pr.md header
- repo @ head SHA

real file contents, not just the patch

UNTRUSTED · AUTHOR-CONTROLLED

- pr.md title + body
- diff.patch
- linked-issue.md
- pr-auditor.md

nonce-fenced, markers defanged

`BEGIN_UNTRUSTED_<9-byte nonce> … END_UNTRUSTED_<nonce>` ← the fence is the boundary

↓

Reviewer agent · Read, Grep, Glob, Edit, Write · --max-turns 40

writes exactly 3 files: review.md · inline-findings.json · recommendation.txt

Rubrics are data: edit rubric.md to change WHAT is checked, the skill to change HOW.

ONE VERDICT, ROUTED BY WHO ACTS NEXT

merge

→ a human merges

changes · block

→ the autonomous fixer

escalate

→ human only: security, secrets, intent

A real silent failure

permission_denials_count: 2 →
it shipped prose with no verdict.
Now: missing verdict ⇒ escalate.

Posted as ONE review with event: COMMENT, never APPROVE or REQUEST_CHANGES. A human always merges.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

The agent’s output is a claim to verify

Two layers of ground truth: what the agent checks, and what CI re-derives without trusting it

LAYER 1 · WHAT THE AGENT CHECKS CLAIMS AGAINST

The checked-out working tree
real file contents at head SHA, not just the patch

TEMPLATE.md
the format spec generated reports must follow

The report’s own tables
every narrative claim must trace to the file’s data

ci-status.md
“CI green” is observed, not assumed

pr-auditor.md
the security verdict is consumed, never re-derived

The rubric
the merge bar is a file on disk, not memory

LAYER 2 · WHAT STAGE 3 RE-DERIVES, IN CODE

Line anchors
re-fetch patches, rebuild anchorable lines, drop off-diff findings

The verdict
4-value whitelist from recommendation.txt; never grepped from prose

Fail-safe
missing, malformed or subverted verdict → escalate

Security tripwire
regex over the review text forces escalate: one-way, in code

Sanitisation
zero-width + bidi Unicode stripped before anything is posted

Seed the verdict as “unset”, not “block”: seeding “block” once hid incomplete reviews as ordinary blocks.

ACT I · THE LOOP LAB

Step 4: close the loop with an autofixer

pr-autofix.yml · the only write-capable job, and the boundary is in code

review: changes / block → round < 3 ? →

Fixer agent · Read, Grep, Glob, Edit, Write

reads the sanitised, nonce-fenced review · no shell, git or gh

SCOPE-LOCK · DETERMINISTIC · THE REAL BOUNDARY

```
every changed path must already be in the PR's diff
never .github/** · .claude/** · skills/** · new files rejected
stage only passing paths, never git add -A
violation or no-op → abort WITHOUT pushing, comment, hand to a human
```

commit "[auto-fix] address PR review (round N+1)" · push · re-dispatch the review

GITHUB_TOKEN pushes can't fire synchronize, so the workflow dispatches the next review explicitly

Its first real catch wasn't the agent. A path-parsing bug in our glue wrote "esults/…".

The scope-lock refused it; nothing was pushed. The boundary protects you from your own code too.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

Every loop needs a counter and a cap

One number, incremented in one place · AUTOFIX_MAX_ROUNDS = 3

R1 — **REVIEW · round = 1**
changes → cap check 1 ≥ 3? no → dispatch fix

fix — **AUTO-FIX · commits "(round 2)"**
push · reply on resolved threads · dispatch review

R2 — **REVIEW · round = 2**
still changes → 2 ≥ 3? no → dispatch fix

fix — **AUTO-FIX · commits "(round 3)"**
push · dispatch review

R3 — **REVIEW · round = 3 · the cap bites**
"Auto-fix stopped: 3-round limit. Handing off to a human."

THE LADDER USUALLY ENDS EARLY

From review

merge → a human merges
escalate → human-only lane
bad verdict / tripwire → escalate

None of these consume a round.

From the fixer

no changes → hand off
scope violation → abort, no push
workflow files ≠ main → skipped

No push, no re-dispatch: the loop stops.

Every exit lands in the same place: a human. Terminal success is "reviewer recommends merge", never an auto-merge.

Real run, PR #21: changes → changes → block → block → block → merge. The fixer corrected "24%" to the real 18%.

ACT I · THE LOOP LAB

What broke, and what held

Real incidents from 31 days. Each one moved a boundary from the prompt into code.

THE AGENT

Confident numbers the tables didn't back

A narrative claimed a "sole 15/15" score that four repos shared. Nothing cross-checked prose.

→ **the reviewer agent now catches it**

THE PERMISSIONS

A review with no verdict

Write was missing; permission_denials_count: 2. Prose shipped, recommendation.txt didn't.

→ **missing verdict fails safe to escalate**

OUR OWN GLUE

"esults/…"

A path-parsing bug dropped one character from a path the fixer had changed.

→ **scope-lock refused it; nothing pushed**

THE TOOLING

--allowedTools is advisory

A later run made 14 Bash calls despite "no shell" in claude-code-action's flags.

→ **safety rests on allowlists + token scope**

If a rule only lives in the prompt, assume it will be broken.

Put it in code that runs after the agent, where the agent can't reach it.

Prompt drift is real too: pr-review/SKILL.md still said "fail safe to block" after the workflow moved to escalate.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT I · THE LOOP LAB

What forms an agent

Four parts, each owned and reviewed separately. The model is the smallest.

THE HOW

Skill

`SKILL.md + scripts/`

Versioned doctrine: what good looks like, what to escalate. Its scripts are run verbatim by CI and by a human locally.

THE JOB

Prompt

`prompts/*.md`

One narrow job. Treats every input as data, never instructions. Reviewed like code: this is where behaviour lives.

THE BAR

Rubric / template

`rubric.md · TEMPLATE.md`

The merge bar and the output format, as data files. Change WHAT is checked without touching HOW it works.

THE CAGE

Workflow

`.github/workflows/*.yml`

Trigger, guards, tokens, and the deterministic stages before and after the model. The only part that holds credentials.

“Change the scoring by editing the skill, not the YAML.”

Separating the parts is what lets a security reviewer, a domain expert and an SRE each own one.

R1 · Generation ≠ verification

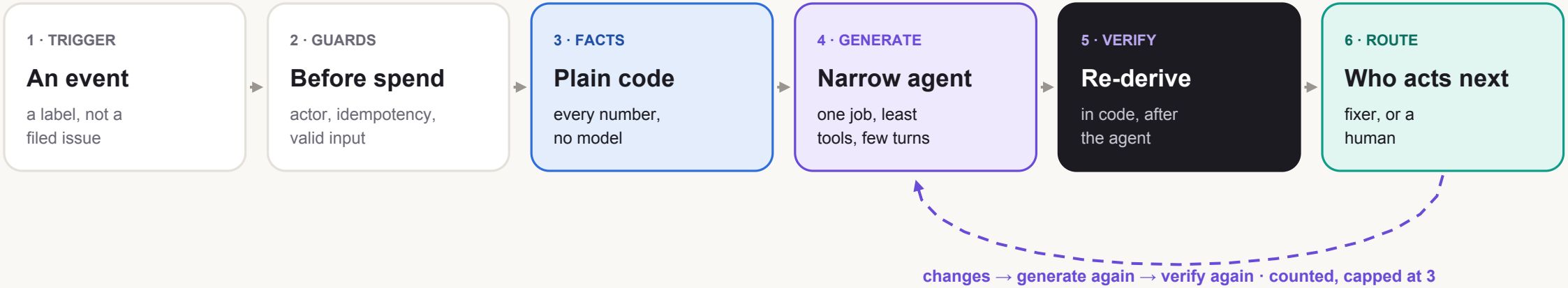
R2 · Bounded blast radius

R3 · Stop and hand back

ACT I · THE LOOP LAB

What forms a loop

Act I’s answer: yes, an agent can work on CI, if the loop around it looks like this



ACT I, IN THREE RULES

R1 · GENERATION ≠ VERIFICATION

Python computes, the agent writes prose; Stage 3 re-derives every claim the reviewer makes.

R2 · BOUNDED BLAST RADIUS

read-only tokens, add-paths, scope-lock, no secrets in the agent’s environment.

R3 · STOP AND HAND BACK

3-round cap, escalate as a lane, draft PRs only, every exit lands at a human.

Next question: does this survive contact with other teams’ production repos?

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT II

Production: the a11y agent

Can the same loop fix real issues in other teams' repos, safely?

2
teams' repos onboarded

32+
agent-authored PRs

44
audit issues, one night

14
decision records

Choosing the first job: bounded, scoped, verifiable

Not every task suits an autonomous contributor. Accessibility fixes pass all three tests.

THE BACKLOG

Hundreds

of open accessibility issues, from audits

Each “owned” by someone on a busy team, so rarely picked up. Meanwhile, WCAG 2.2 AA and the European Accessibility Act raise the bar.

The missing capability was throughput, not knowledge.

Automated tools catch roughly a third to a half of WCAG issues: we promise “resolves detectable violations”, never “compliant”.

Bounded

√

one component, one WCAG rule, one PR

`triage sizes it: trivial · moderate · complex`

Path-scoped

√

front-end markup, style and ARIA folders only

`path_allowlist, checked before and after`

Tool-verifiable

√

axe, contrast and unit tests exit non-zero on failure

`the gate re-runs them; the agent’s word is not evidence`

Escalated, never guessed

design-system defects · meaningful alt text · copy changes · anything outside the allowlist → a question for a human

Architecture: one central agent, many target repos

The agent never runs in a team's CI. Teams hold config, not credentials.

Issue form + label

filing is free; a maintainer with
write access applies
ready-for-development to start

Project v2 board

Backlog → Ready → In progress
→ In review → Done
one writer per transition
status IDs self-checked by a smoke

AUTONOMOUS-CODING-WORKFLOWS (CENTRAL)

`a11y-agent-run.yml`	~4,000-line staged workflow
`prompts/ · skills/ · actions/`	reviewed like code
`vendor/modern-web-guidance`	pinned + checksummed
`telemetry → control plane`	deterministic collector
`Bedrock via OIDC`	no static model keys

PR

Target repo (×2)

`CLAUDE.md · .a11y-agent/config.yml`
`baseline.json · merged-callback.yml`
`a11y-fix-relay.yml`
`branch protection, no App bypass`

Doorbells, not keys

target repos hold a dispatch-only App:
actions: write on the central repo,
nothing else. They forward IDs only;
the central handler re-checks all.

THE AGENT'S OWN GITHUB APP · TOKENS MINTED PER RUN, ~1 HOUR, SCOPED TO THE TARGET REPO

contents: write	pull_requests: write	issues: write	projects: write
no admin	no workflows	no merge	no secrets

Install scope ≠ action scope.

Installed org-wide so dependency installs resolve; it acts only where onboarded.

ACT II · PRODUCTION: THE a11y AGENT

One run, stage by stage

a11y-agent-run.yml · two model sessions inside four deterministic stages

STAGE 1 · CI

Parse + guard

`no model`

issue form parsed by Node, never interpolated in shell
repo + paths regex-checked
requested paths ⊂ allowlist
board → In progress

STAGE 2a · AGENT

Triage

`Sonnet 5.5 · 24 + 45 turns`

read-only: no Bash, no Edit, no network
complexity × ownership
JSON verdict + phased plan
checkout asserted byte-clean

CI

Rebuild plan

`no model`

fixed key list only
files ⊂ allowlist
WCAG refs format-checked
plan posted on the issue before any code is written

STAGE 2b · AGENT

Implement

`Opus 5.5 · 140 turns`

executes the agreed plan
one commit per phase
deps pre-installed by CI
no telemetry keys in env
opens the PR

STAGE 3 · CI

Gate

`no model`

refresh baseline from the merge-base
re-run the team's commands
allowlist vs real diff
fail → PR to draft

escalate → a successful run

comments the blocking question;
no PR, ~a third of the cost

Review → "@a11y-agent-fix" → fix mode

relay forwards IDs only · ≤20 threads · 60 turns ·
one commit per thread · same gate again

WHY SPLIT TRIAGE FROM IMPLEMENTATION

Escalation becomes deterministic

a parsed JSON verdict, not prose to interpret

Cost

learning "this is a design-system bug" is cheap

Blast radius

the session reading the most untrusted text has no tools

R1 · Generation ≠ verification

R2 · Bounded blast radius

R3 · Stop and hand back

Trust boundaries: what each stage can read, do and reach

The issue body is data, never instructions, and the architecture enforces it rather than the prompt

STAGE	UNTRUSTED INPUT IT READS	WHAT IT CAN DO	CREDENTIALS IN REACH
Parse (Node)	issue body, as form fields	board moves, checkout	**App token, 1 h, one repo**
Triage (Sonnet)	issue + target repo	read files only; write a verdict	**model access only**
Plan rebuild (Node)	triage's JSON	copy a fixed key list	**none**
Implement (Opus)	rebuilt plan + repo	edit, build, test, commit	**no telemetry / long-lived keys**
Gate (CI)	the pushed branch	run the team's own commands	**baseline-writer opt-in var**
Metrics (separate job)	artifacts, type-checked	publish numbers, never content	**Datadog key lives only here**

Remember Act I: --allowedTools is advisory.

What actually holds: the path allowlist on the real diff, token scope, and what is absent from the environment.

Onboarding is a contract the team owns

Four files in the target repo. The agent must not edit any of them; the gate rejects any diff that tries.

```
# performance-analytics/.a11y-agent/config.yml (excerpt)

install_cmd:   bash scripts/a11y/ci-install.sh
build_cmd:   yarn turbo run dist …storybook
serve_cmd:   node scripts/a11y/serve.mjs & wait-for
test_cmd:   yarn turbo run test --filter=./packages/*
contrast_cmd:   node scripts/a11y/contrast.mjs
checker:   axe
max_violations:   0
min_contrast_normal:   4.5
viewports:   [375x812, 768x1024, 1280x800]
scan_targets:   - …iframe.html?id=filterscomposer--default
path_allowlist:
   - apps/insights/src/components
   - packages/insights-shared/src
   - packages/visualizations/src/components
   - … 11 entries
```

THE FOUR FILES

`CLAUDE.md`
how the repo works, in prose, for agent and humans

`.a11y-agent/config.yml`
the commands that prove a fix works

`merged-callback.yml`
doorbell: merged → card to Done

`a11y-fix-relay.yml`
doorbell: reviewer asks for a fix

The allowlist is the team's blast radius

In the 44-issue audit, 13 issues touched folders like visualizations/src/hooks. Rejected at Stage 1, before any model ran: $0, and a question for the team.

The gate: evidence over assertion

Stage 3 re-runs the team's own commands on the pushed branch, then attributes every failure

CHECKS, EACH WITH ITS OWN RECORDED OUTCOME

ALLOWLIST	real PR diff ⊂ path_allowlist
PROTECTED	.a11y-agent/ · CLAUDE.md untouched
A11Y_CMD	axe on scan_targets × viewports
CONTRAST_CMD	4.5 : 1 normal · 3 : 1 large
TEST_CMD	unit tests for changed workspaces
MARKER + BASE	issue marker, default branch
BASELINE	matches main or the fresh copy

EVERY FAILURE IS ATTRIBUTED

BLOCKING

the agent's change caused it → PR converted to draft

with the per-command table posted on the PR

INFRA

the environment failed → PR left untouched

e.g. GRAPHQL_SCHEMA_URL unset; regate later for free

ADVISORY

pre-existing or stale → noted, not blamed

never let off for re-breaking what main just fixed

Only tool output moves a card to In review.

`regate · gate_only · dry_run: re-run Stage 3 alone, zero model spend.`

ACT II · PRODUCTION: THE a11y AGENT

Keeping the judge honest: the baseline refresh

ADR-0009 · known violations are subtracted, so the list must describe main as it is now

THE PROBLEM: MAIN MOVES SEVERAL TIMES A DAY

main adds a violation

→ the agent's PR blocks on code it never touched

main fixes a violation

→ its stale allowance silently forgives a regression

THE CONSTRAINT

The agent must never regenerate the file that decides whether its own work passed. --update-baseline refuses under CI, and the gate rejects agent diffs to .a11y-agent/.

1 · MERGE-BASE

check out the merge-base of branch and main, never the branch

2 · REFRESH

build, serve, scan; write the baseline (workflow, not agent)

3 · JUDGE

check out the branch, rebuild, scan against the fresh copy

4 · PRUNE

entries the branch fixed may be removed; additions are discarded

5 · COMMIT

separate gate commit with a trailer; content re-asserted, not trusted

```
chore(a11y): refresh baseline from main 3f9c2e1 A11y-Baseline-Source: 3f9c2e1
```

Writer only runs if A11Y_BASELINE_REFRESH == git rev-parse HEAD; the agent's environment never carries it.

No timestamps in the file, so two PRs that saw the same drift merge cleanly. A failure on main is INFRA, never blamed on the agent.

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT II · PRODUCTION: THE a11y AGENT

How it got here: real failures, one decision record each

Every incident became a mechanism in code, then an ADR. An instruction alone never stuck.

AUG 24 · ADR-0001

Prove the plumbing with the model off

board, logins and 5 transitions, zero agent runs: 4 silent setup bugs found, incl. swapped IDs that fail with no error

→ a permanent smoke test

AUG 31

Triage spent 46 turns, $3.50, no verdict

“be concise” in the prompt did nothing. A provisional pass must save a verdict by turn 10; refine runs on a separate cap

→ a turn cap is a mechanism

SEP 11 · ADR-0007

Blamed for 9 violations it never touched

issue #35: a PR drafted for a component outside its diff. And a hidden bug meant tests never ran at all

→ per-command outcomes + tiers

SEP 18

Fixed at turn 91, cap was 90

~40% of turns went on Storybook rebuilds and probe scripts. Setup moved into CI; the agent stopped chasing the unverifiable

→ prep out of the model’s budget

SEP 28 · ADR-0009

The known-issues list went stale

a reviewer asked: does the agent regenerate the baseline? It must not, so the gate does, from the merge-base

→ fresh judge, every run

SEP 29

GitHub silently dropped queued runs

labelling several issues at once cancelled pending runs in one concurrency group; a dedup guard tripped on its own message

→ per-issue queues; serial batches

R1 · Generation ≠ verification

R2 · Bounded blast radius

R3 · Stop and hand back

ACT II · PRODUCTION: THE a11y AGENT

One run, measured from the outside

performance-analytics #154 · gate passed · every number collected after the agent, never self-reported

TIME TO COMPLETION	TURNS	COST	CACHE HIT	WARNINGS
34m 36s	174	$7.17	97.6%	0

STAGE TIMELINE



AGENT SESSIONS

Triage · provisional

`sonnet-5 · stage 2a`

$1.45	39 / 24
cost	turns / cap

cache hit 95.3% · 42 responses

Triage · refine

`sonnet-5 · stage 2a`

$0.96	25 / 45
cost	turns / cap

cache hit 92.7% · 14 responses

Implementation

`opus-5 · stage 2b`

$4.77	110 / 140
cost	turns / cap

cache hit 98.7% · 110 responses

What the numbers told us

Triage is a third of the spend. And the dashboard flagged the provisional pass at 39 turns against a cap of 24: measure your caps, don't assume them.

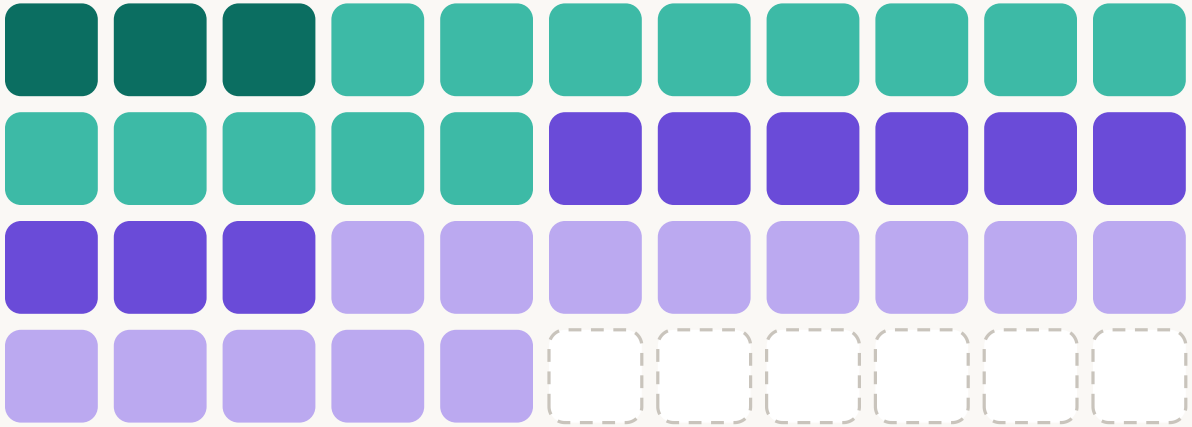
R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

ACT II · PRODUCTION: THE a11y AGENT

A full audit, burned down overnight

44 accessibility issues from the Performance Analytics audit, filed on Oct 6, worked through by the next morning

EVERY SQUARE IS ONE ISSUE FROM THE AUDIT



16
pull requests
- 3 ready for review
- 13 drafts, waiting on a check

22
flagged for a person
- 9 by AI triage, with a question
- 13 out of scope, caught for $0

6
didn't finish
- stopped short of a PR; retry or rescope

86% — **38 of 44 handled overnight**
Each one became a fix or a clear question for a person

THE SAME WORK, TWO WAYS

	AGENT	CONTRACTOR (EST.)
Time	**18 h** overnight, unattended	**~3 weeks** 14.5 working days
Cost	**$192** AI cost, all 44 issues	**~€7,250** at €500 a day

Both still go through the same accessibility-team review.

ESTIMATED SAVING

95%+ cheaper, and done by the next morning

Source: Performance Analytics audit database and 45 agent runs, Oct 6–7, 2026 (one at a time, 14:27 → 08:41). Contractor estimate: 1 h to triage each of the 44 issues, plus ½ day per small fix and 1 day per medium fix for the same 16 PRs (14 small, 2 medium) = 116 h ≈ 14.5 days at €500/day. 9 more audit gaps were already filed and were skipped.

ACT II · PRODUCTION: THE a11y AGENT

Act II: the same three rules, now load-bearing

What changed from the lab: real teams, real reviewers, other people’s code

R1 · GENERATION ≠ VERIFICATION

The team’s tools judge

triage verdict re-validated
plan rebuilt from fixed keys
gate re-runs team commands
failures attributed by cause
baseline refreshed by CI
metrics collected after the run

R2 · BOUNDED BLAST RADIUS

Config, not credentials

App: no admin, merge, workflows
1-hour tokens, one repo
doorbell Apps in target repos
allowlist checked twice
agent can’t edit its config
no telemetry keys near models

R3 · STOP AND HAND BACK

Asking is a success

escalation is a green run
plan visible before code
draft on any BLOCKING check
turn caps per session
review-fix capped per thread
a human merges every PR

Honest scorecard from an audit: 6 of 44 issues didn’t finish, and 13 PRs are drafts.

Both are the system working: a run that can’t prove its work stops and says so, instead of shipping a plausible PR.

R1 · Generation ≠ verification
R2 · Bounded blast radius
R3 · Stop and hand back

ACT III · THE PLATFORM

From 1 agent to 4 in one week

The a11y agent's operating model meant every new agent started from a working one

MON SEP 28	TUE SEP 29	WED SEP 30	THU OCT 1	FRI OCT 2	SAT OCT 3	SUN OCT 4
1 **a11y agent** Susanna Wong — Already live; second team's fixes landing	2 **Feature flags** Andy Kuszyk — Reviews flags in LaunchDarkly; files clean-up issues	Shared rules extended for org-wide changes	3 **Modernizer** Giorgio Pellero — Upgrades code across every repo; ran the next day	4 **MCP feedback** Adrian Carayol — Triages customer feedback; built in one day	**+ Dashboard** Control plane — One view of every run, for all four agents	

THE FOUNDATION THE a11y AGENT BUILT · EVERY NEW AGENT GETS IT FOR FREE

Request form	Robot account	Tracking board	Quality checks	Safety rules	Metrics
a safe front door	short-lived keys	status for free	evidence, not claims	reviewed once	cost and time per run

5 engineers · **4 agents** · **20 recorded decisions** · **22 agent PRs merged** in one shared home

ACT III · THE PLATFORM

A second archetype: the general-purpose agent

Modernizer · an operator describes a change; the agent finds every repo it applies to

PURPOSE-BUILT · a11y agent

Domain knowledge lives in the skills.
One class of issue, many repos, onboarded one by one.

GENERAL-PURPOSE · Modernizer

The campaign issue carries the intent.
Any pattern: library upgrades, replacements, extractions.

CAMPAIGN
Issue + label
the change, in words
maintainer starts it

→

AGENT · DISCOVER
Discoverer
read-only org token
CI validates, dedupes,
files target issues

→

AGENT · PLAN
Planner
read-only checkout
per-repo plan rendered
into the issue

→

HUMAN
Approve
modernizer-approved
label, per repo

→

AGENT · EXECUTE
Executor
commits locally,
never holds a write
token

→

CI
Gate → draft PR
rebase, verify,
push, open draft

THE MAINTAIN LOOP · MODERNIZER-SWEEP.YML, EVERY 5 MINUTES

```
rebase when behind or conflicting · ≤ every 2 h, ≤ 6 a day
CI fix only when checks have settled red (nothing pending, 3 min)
answer review feedback · max 3 fix runs a day, then needs-human
out-of-plan fixes kept only if a check fails without them and passes with them
```

FIRST CAMPAIGN · OCT 2

18 / 19

repos merged a fix for the same LaunchDarkly SDK panic

R1 · Generation ≠ verification
R2 · Bounded blast radius
R3 · Stop and hand back

ACT III · THE PLATFORM

The control plane: measured from the outside

ADR-0013/0014 · the agent never reports on itself, and never holds the credentials to publish

Agent job finishes (or crashes)
`execution_file per session · Jobs API timings`

↓

Deterministic collector · if: always()
`counts, durations, cost, tokens, tool names`

↓

run-metrics.json · schema v2
`numbers and labels only · never content`

↓

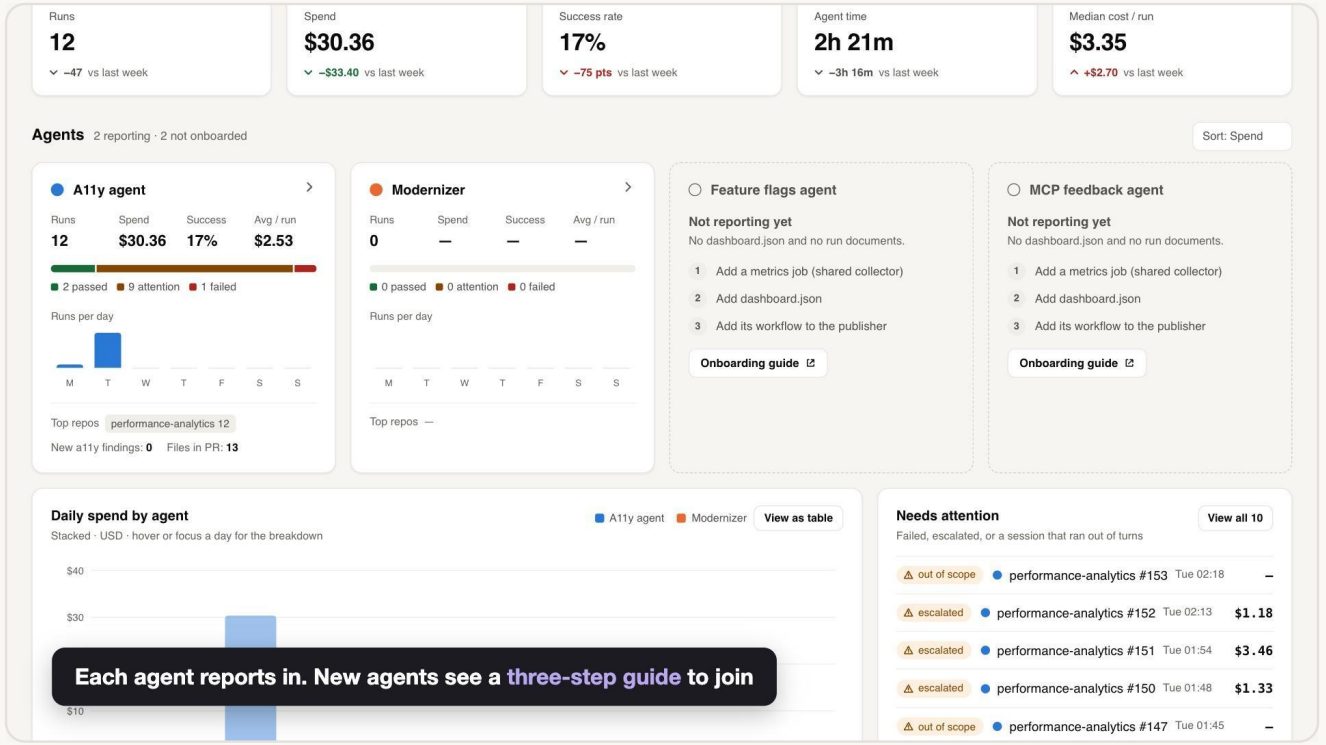
Publisher workflow · workflow_run
`validates the contract · write-only OIDC role`

↓

S3: one immutable doc per run attempt
`derived index files, rebuildable`

↓

Static React app · CloudFront + Okta
`no API server · drill-down to each turn`



Crashed, timed-out and cancelled runs still get measured

No Datadog or S3 credential ever shares a runner with a model

Onboarding an agent: one PR, one dashboard.json

R1 · Generation ≠ verification

R2 · Bounded blast radius

R3 · Stop and hand back

ACT III · THE PLATFORM

Impact so far

Ttwo months in, as of Oct 12, 2026

4

agents

two purpose-built, one general-purpose, one triage

5

engineers building

on one shared repo, rules and gate

21

repos with agent PRs

2 onboarded for a11y, 19 reached by one campaign

55

agent-authored PRs

a11y and Modernizer, drafts included

22

merged by engineers

4 accessibility fixes, 18 campaign fixes

$192

for a 44-issue audit

overnight, unattended; ~3 contractor-weeks

1 day

to a new agent

Modernizer's first campaign ran the day after its build

0

merged by an agent

by design, every time

Sources: GitHub PR search (Oct 12), Performance Analytics audit runs (Oct 6–7), git history. 55 = 32 + 4 early a11y PRs + 19 Modernizer.

ACT III · THE PLATFORM

Next: from agents to a software factory

Audit agents find the work, specialist and general agents do it, one gate checks it, people decide

FRIDAY NIGHT · NEXT

Audit agent

scans a codebase, files issues

THE WEEKEND

Fix agents

triage and fix what they can

MONDAY

People review

fixes to approve, questions

EVERY MONTH

Repeat

the backlog never rebuilds

CONTROL PLANE

every run, every cost, every escalation, across every agent

INTAKE

audit findings
customer feedback
upgrade campaigns
flag clean-up
team requests

AGENTS

- Accessibility
- Modernizer
- Feature flags
- MCP feedback
- + the next archetype

ONE GATE

in bounds
team's tests
evidence only
no self-grading

PEOPLE

review
merge
ship

When do we loosen a rule? When the shadow-merge rate says so: the share of agent PRs that would have merged with zero human edits.

R1 · Generation ≠ verification

R2 · Bounded blast radius

R3 · Stop and hand back

The operating model, on one slide

Take a photo. Every item here is something that broke without it.

RULE 1

Generation ≠ verification

- ☐ Plain code computes every number; the model writes prose
- ☐ Re-derive the agent's claims after it runs: anchors, verdicts, diffs
- ☐ The team's own commands are the judge, and their baseline is fresh
- ☐ Measure runs from outside the agent, including the ones that crash
- ☐ Seed verdicts as "unset"; missing evidence means escalate
- ☐ Rubrics and templates are data, reviewed like code

RULE 2

Bounded blast radius

- ☐ Start on an event a write-access human applies, never on filing
- ☐ Short-lived tokens, one repo, no merge, admin or workflow scope
- ☐ Path allowlist on the request and again on the real diff
- ☐ The agent can't edit its own config, rules, baseline or workflows
- ☐ No long-lived secret in a model's environment, ever
- ☐ Assume tool flags are advisory; enforce in code after the run

RULE 3

Stop and hand back

- ☐ Separate triage that can say "this needs a human"
- ☐ Escalation is a successful run, not a red one
- ☐ Turn caps per session, round caps per loop, counted in one place
- ☐ Show the plan before the code so anyone can stop it
- ☐ Draft on any blocking check; never a plausible PR
- ☐ Every exit lands at a human, and a human merges

R1 · Generation ≠ verification | R2 · Bounded blast radius | R3 · Stop and hand back

THE ONE THING TO TAKE HOME

Autonomy is a property of the system, not the model.

The model generates | Code verifies | A human decides

Thank you

Susanna Wong · Staff AI Engineer, Typeform

Questions?

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

